Binary Search Tree Operations

```
#include <stdio.h>
#include <stdlib.h>

struct TreeNode {
	int val;
	struct TreeNode* left;
	struct TreeNode* right;
};




struct TreeNode* createNode(int val) {
	struct TreeNode* newNode = (struct TreeNode*)malloc(sizeof(struct TreeNode));
	newNode -> val = val;
	newNode -> left = NULL;
	newNode -> right = NULL;
	return newNode;
}

struct TreeNode* insertNode(struct TreeNode* root, int data){
	if(root == NULL){
		return createNode(data);
	}
	if(data < root -> val){
		root -> left = insertNode(root -> left, data);
	}
	else if(data > root -> val){
		root -> right = insertNode(root -> right, data);
	}
```

```
	return root;
}

void inorder(struct TreeNode* root) {
	if (root == NULL) return;
	inorder(root -> left);
	printf("%d ", root -> val);
	inorder(root -> right);
}

void inorderTraversal(struct TreeNode* root){
	if(root == NULL){
		printf("The tree is empty\n");
		return;
	}
	inorder(root);
	printf("\n");
}

void preorder(struct TreeNode* root) {
	if(root == NULL) return;
	printf("%d ", root -> val);
	preorder(root -> left);
	preorder(root -> right);
}

void preorderTraversal(struct TreeNode* root){
	if(root == NULL){
		printf("The tree is empty\n");
		return;
	}
```

```
	preorder(root);
	printf("\n");
}

void postorder(struct TreeNode* root) {
	if(root == NULL) return;
	postorder(root -> left);
	postorder(root -> right);
	printf("%d ", root -> val);
}

void postorderTraversal(struct TreeNode* root){
	if(root == NULL){
		printf("The tree is empty\n");
		return;
	}
	postorder(root);
	printf("\n");
}

struct TreeNode* findMin(struct TreeNode* root){
	while(root -> left != NULL) {
		root = root -> left;
	}
	return root;
}

struct TreeNode* deleteNode(struct TreeNode* root, int key, int *found) {
	if(root == NULL){
		return NULL;
	}
```

```
	if(key < root -> val){
		root -> left = deleteNode(root -> left, key, found);
	}
	else if(key > root -> val){
		root -> right = deleteNode(root -> right, key, found);
	}
	else{
		*found = 1;
		if(root -> left == NULL){
			struct TreeNode* temp = root -> right;
			free(root);
			return temp;
		}
		else if(root -> right == NULL){
			struct TreeNode* temp = root -> left;
			free(root);
			return temp;
		}
		struct TreeNode* temp = findMin(root -> right);
		root -> val = temp -> val;
		root -> right = deleteNode(root -> right, temp -> val, found);
		}
	return root;
}

void freeTree(struct TreeNode* root){
	if(root == NULL)
		return;
	freeTree(root -> left);
	freeTree(root -> right);
	free(root);
```

```
}


int main() {
    struct TreeNode* root = NULL;
    int choice, data;

    while (1) {
        printf("1. Insert Node\n");
        printf("2. In-Order Traversal\n");
        printf("3. Pre-Order Traversal\n");
        printf("4. Post-Order Traversal\n");
        printf("5. Delete Node\n");
        printf("6. Exit\n");
        printf("Choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Data: ");
                scanf("%d", &data);
                root = insertNode(root, data);
                break;
            case 2:
                inorderTraversal(root);
                break;
            case 3:
                preorderTraversal(root);
                break;
            case 4:
                postorderTraversal(root);
```

```
            break;
        case 5: {
            printf("Delete: ");
            scanf("%d", &data);
            int found = 0;
            root = deleteNode(root, data, &found);
            if (!found) {
                printf("Value not found\n");
            }
            break;
        }
        case 6:
            freeTree(root);
            exit(0);
        default:
            printf("Invalid choice\n");
    }
}

return 0;
}
```